

Week04 | 实验 | 最小 Iceberg 闭环: catalog、warehouse、Bronze/Silver、time travel

本页结构

先把 Week04 的最小 Lakehouse 闭环跑通, 再谈更重的扩展	1
这次实验要完成什么	1
参考学习时间	2
本次实验产出	2
先看一张闭环图	2
与 OmniSupport Copilot Week04 实现对齐	2
前置条件	3
本次实验真正验证哪些不变量	4
0. 命令约定	4
Step 0: 确认 Week3 ingest 数据已就绪	4
Step-by-step 总览	5
Step 1: 确认 Week4 catalog / env 配置	5
Step 2: ensure namespace / tables	5
Step 3: materialize 4 表	6
Step 4: inspect snapshots / history / files	6
Step 5: 生成第二个 snapshot	7
Step 6: time travel	7
Step 7: schema evolution add-column	8
Step 8: 生成 baseline report	8
常见错误与排查表	8
重跑与清理说明	9
最后你应该得到什么	9
自检清单	9
课后最小行动	9

先把 Week04 的最小 Lakehouse 闭环跑通, 再谈更重的扩展

这次实验不要求你在学生本地搭一整套重型湖仓平台。

你要验证的是:

当前项目是否已经具备 catalog、warehouse、最小 4 表、history / snapshots 和 time travel 的最小闭环。

这次实验要完成什么

本次实验不是在项目外另造一套 lakehouse demo。

它要直接围绕当前 Week04 基线对象来验证:

- catalog 能否正常加载;
- warehouse 是否可写;
- `bronze.raw_ticket_event` 是否能 materialize;

- `bronze.raw_doc_asset` 是否能 materialize;
- `silver.ticket_fact` 是否能 materialize;
- `silver.knowledge_doc` 是否能 materialize;
- history / snapshots / files 能否查看;
- time travel 是否能演示;
- schema evolution add-column 是否能留下记录;
- baseline report 是否能写出可复核结论。

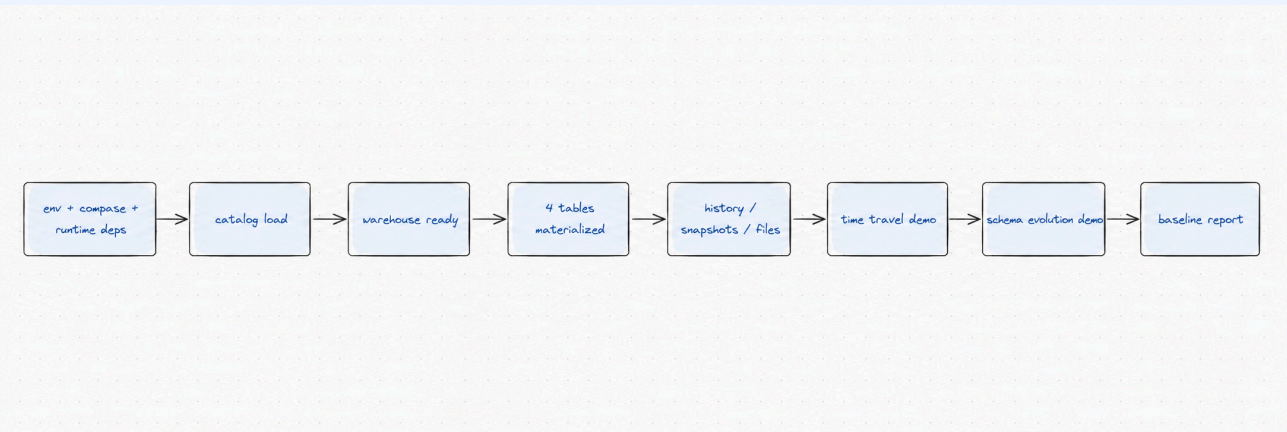
参考学习时间

建议按一次完整实验安排：先跑通最小闭环，再整理观察记录和 baseline report。

本次实验产出

- catalog smoke 输出记录
- `reports/week04/materialization_report.json`
- `reports/week04/time_travel_demo_report.md`
- `reports/week04/schema_evolution_demo_report.md`
- `reports/week04/iceberg_baseline_report.md`
- 对 `runbooks/week04/README.md` 的执行批注

先看一张闭环图



与 OmniSupport Copilot Week04 实现对齐

本页已对齐项目仓库 `dataPro-lgtm/omnisupport-copilot` 的 Week04 实现。学生操作时不要另起一套 demo，也不要继续把下面命令当占位结构。

真实同步锚点如下：

类型	项目路径	本实验怎么用
主 runbook	<code>runbooks/week04/README.md</code>	命令顺序以它为准
课程同步包	<code>docs/blueprints/week04/course_site_sync_packet_v1.md</code>	防止课程页与项目命令漂移
配置检查	<code>pipelines/lakehouse/settings.py</code>	检查 catalog / warehouse / MinIO env

类型	项目路径	本实验怎么用
Catalog smoke	<code>pipelines/ lakehouse/catalog.py</code>	加载 SQL Catalog、确保 bucket / namespace / 4 表
物化脚本	<code>pipelines/ lakehouse/materialize.py</code>	dry-run 与真实写入 4 张核心表
Metadata inspection	<code>pipelines/ lakehouse/inspect_metadata.py</code>	查看 snapshots / history / files / metadata-log
Time travel	<code>pipelines/ lakehouse/demo_time_travel.py</code>	生成 time travel demo report
Schema evolution	<code>pipelines/lakehouse/ demo_schema_evolution.py</code>	只演示 add-column
Baseline report	<code>pipelines/ lakehouse/perf_baseline.py</code>	生成 Week04 验收基线

前置条件

开始前先确认：

- Week03 ingest baseline 已经跑通，至少有 ticket / document 的可用输入；
- 项目环境能通过 Docker Compose 启动；
- PostgreSQL 与 MinIO 能访问；
- 你能定位 Week04 相关代码位置；
- 你知道当前命令以项目仓库真实实现为准。

需要定位的 repo 对象：

- `infra/docker-compose.yml`
- `pyproject.toml`
- `pipelines/lakehouse/iceberg_schemas.py`
- `pipelines/lakehouse/materialize.py`
- `pipelines/lakehouse/inspect_metadata.py`
- `runbooks/week04/README.md`
- `docs/blueprints/week04/source_to_iceberg_mapping_v1.md`

前置项	你要确认什么	如果不满足先做什么
Docker Compose 环境	PostgreSQL、MinIO、devbox / app service 能启动	回到项目 README / runbook 修环境
<code>.env.local</code>	catalog uri、warehouse、S3 endpoint、凭证齐全	不要在命令里硬编码
Week3 ingest 数据	ticket / document 输入可解释	先补 Week3 lab 输出
Week4 脚本	项目仓库已有真实 runbook / module	以 <code>runbooks/week04/README.md</code> 和 <code>course_site_sync_packet_v1.md</code> 为准
MinIO / PostgreSQL	本机与容器内访问路径清楚	分清 host endpoint 与 container endpoint

本次实验真正验证哪些不变量

不变量	怎么验证	失败说明什么
catalog 可加载	catalog smoke report / name-space 可列出	SQL Catalog 配置或网络不通
warehouse 可写	create / materialize 后能看到 metadata 与 data files	MinIO endpoint、bucket、凭证或 table location 错
4 表存在	ensure namespace / tables 输出	schema source of truth 没对齐
至少 2 个 snapshot	追加、重跑或 add-column 后 inspect	只有 create table, 没有形成状态变化
旧 snapshot 可读	time travel demo	snapshot 引用错或保留策略不允许
baseline report 可生成	markdown report 包含指标与结论	只是跑命令, 没有沉淀验收证据

0. 命令约定

下面的命令都在 `omnisupport-copilot` 项目仓库根目录执行。Week04 的学生主路径是 Docker devbox, 不要求宿主机直接安装 PyIceberg / PostgreSQL / MinIO。

```
cp infra/env/.env.example infra/env/.env.local
```

```
docker compose --env-file infra/env/.env.local -f infra/docker-compose.yml \
  up -d --build postgres minio minio_init
```

如果你已经有 `.env.local`, 不要盲目覆盖。先对照 `infra/env/.env.example` 补齐 Week04 的 `ICEBERG_*` 与 `WEEK04_*` 配置。

Step 0: 确认 Week3 ingest 数据已就绪

目标: 确认 Week04 不是凭空造数据, 而是接住 Week03 的 ingest baseline。

你要检查

- ticket source 是否已有可用输入;
- document source 是否已有可用输入;
- Week03 manifest / state / report 是否能解释输入边界。

期望输出

- 有明确 source coverage;
- 能说明本次 Week04 materialization 消费哪批输入;
- 如果输入为空, 先不要继续做 Iceberg 演示。

失败排查

- 回到 Week03 lab;
- 检查 manifest;
- 检查 contract tests;

- 检查 seed_loader dry-run 输出。

Step-by-step 总览

步骤	目的	期望输出	失败先看什么
0. 确认 Week3 数据	证明输入边界	source coverage / manifest / state	Week3 lab 与 dry-run
1. 检查 catalog env	证明配置可加载	catalog smoke report	.env.local / PostgreSQL
2. ensure name-space / tables	证明最小 4 表存在	bronze / silver 表清单	schema source of truth
3. materialize	证明真实写入	row count / snapshot	mapping / dedupe
4. inspect	证明状态记忆	snapshots / history / files	catalog 指向与 warehouse
5. 第二个 snapshot	证明可形成状态变化	snapshot count 增加	写入是否真正发生
6. time travel	证明旧状态可读	old vs current 对比	snapshot id
7. add-column	证明演进边界	schema id / history	nullable 与兼容性
8. baseline report	证明可交接	markdown report	是否只有截图无解释

Step 1: 确认 Week4 catalog / env 配置

目标：确认 PostgreSQL-backed SQL Catalog 与 MinIO warehouse 的配置边界清楚。

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
python -m pipelines.lakehouse.settings --check
```

期望输出

- catalog 可连接；
- warehouse bucket / path 可访问；
- host endpoint 与 container endpoint 没有混用；
- 配置项能写入 runbook。

验收不变量

catalog 可加载，但 warehouse 不能写，不算通过。

Step 2: ensure namespace / tables

目标：确保最小 4 表存在，并且 schema 与项目 source of truth 对齐。

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
python -m pipelines.lakehouse.catalog --smoke
```

期望输出

- bronze.raw_ticket_event 存在；

- `bronze.raw_doc_asset` 存在；
- `silver.ticket_fact` 存在；
- `silver.knowledge_doc` 存在；
- schema 来源能追溯到 `pipelines/lakehouse/iceberg_schemas.py` 或项目真实 DDL。

失败排查

- schema 字段名不一致；
- namespace 未创建；
- catalog 指向错；
- table location 与 warehouse 不一致。

Step 3: materialize 4 表

目标：从 Week03 baseline 输入生成最小 Bronze / Silver 表状态。

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.lakehouse.materialize --all-core --dry-run
```

确认 dry-run row count 正常后，再写入 Iceberg：

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.lakehouse.materialize --all-core \
    --report-json reports/week04/materialization_report.json
```

期望输出

- 每张表至少形成一次可解释写入；
- row count 与 source coverage 能对上；
- 重跑不会产生不可解释重复；
- materialization report 能说明本次写入影响。

验收不变量

只 create table、没有真实 materialization，不算完成实验。

Step 4: inspect snapshots / history / files

目标：证明表状态记忆真实存在。

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.lakehouse.inspect_metadata --table silver.ticket_fact --view snapshots
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.lakehouse.inspect_metadata --table silver.ticket_fact --view history
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.lakehouse.inspect_metadata --table silver.ticket_fact --view files
```

如需检查 metadata log:

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
  python -m pipelines.lakehouse.inspect_metadata --table silver.ticket_fact --view metadata-log
```

期望输出

- snapshot count;
- latest snapshot time;
- file count;
- avg / min / max file size;
- partition distribution;
- metadata log entry count (如脚本支持)。

失败排查

- 表没有形成新 snapshot;
- warehouse 写入路径异常;
- inspect 命令指向了另一个 catalog;
- history 最新时间早于本次实验。

Step 5: 生成第二个 snapshot

目标: 证明 Week04 不是只有一个初始状态, 而是能形成可回看的状态变化。

建议动作

- 对同一 source 做一次可控追加;
- 或执行一次 add-column schema evolution;
- 或在项目支持时做一次小规模 materialization rerun。

期望输出

- 至少一张表的 snapshot count 增加;
- history 能显示新提交;
- baseline report 记录变化原因。

Step 6: time travel

目标: 证明可回看不是空话。

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
  python -m pipelines.lakehouse.demo_time_travel \
  --table silver.ticket_fact \
  --out reports/week04/time_travel_demo_report.md
```

期望输出

- 能指定旧 snapshot;
- 能读到旧状态;
- 能说明旧状态与当前状态差异;
- demo output 写入 time_travel_demo_report.md。

Step 7: schema evolution add-column

目标：演示最小 schema evolution 边界。

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.lakehouse.demo_schema_evolution \
  --table bronze.raw_doc_asset \
  --add-column source_checksum_algo \
  --out reports/week04/schema_evolution_demo_report.md
```

期望输出

- schema 增加新列；
- history / snapshots 有可解释记录；
- 老数据仍可读；
- 说明为什么本周只做 add-column，不做 rename/drop/type update。

Step 8: 生成 baseline report

目标：把实验结果整理成可交接证据。

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.lakehouse.perf_baseline \
  --all-core \
  --out reports/week04/iceberg_baseline_report.md
```

至少写入：

- 运行环境；
- 目标表清单；
- row count；
- snapshot count；
- file count；
- file size 分布；
- partition distribution；
- time travel demo 结论；
- schema evolution demo 结论；
- 已知问题与下一步建议。

常见错误与排查表

错误	快速判断	排查方向
catalog 连接失败	namespace 无法列出	PostgreSQL catalog 配置
warehouse 写失败	表注册成功但文件不可见	MinIO endpoint / bucket / credential
host 与 container endpoint 混用	本机能跑，容器失败	Docker 网络与环境变量

错误	快速判断	排查方向
schema 不一致	append 报字段或类型错误	<code>iceberg_schemas.py</code> 与 <code>source schema</code>
重跑后重复	row count 异常增长	dedupe key / idempotency
time travel 失败	找不到旧 snapshot	snapshot 保留与 inspect 输出
baseline report 为空	命令跑过但没有采集指标	检查 report 生成步骤和 inspect 输出
页面命令与 runbook 不一致	学员照页面跑失败	以项目仓库 Week4 runbook 为准，课程页同步更新

重跑与清理说明

本周不要求做完整 cleanup pipeline，但你必须在 runbook 里说明：

- 哪些动作可以安全重跑；
- 哪些动作会产生新 snapshot；
- 哪些表可以删除重建；
- 哪些状态必须保留给 baseline report；
- 如果要清理 MinIO / catalog，先备份哪些报告。

最后你应该得到什么

实验结束时，你至少应该能交出：

- 一次可复现的 Week04 闭环；
- 一份 catalog smoke 输出记录；
- 一份 `reports/week04/materialization_report.json`；
- 一份 time travel 记录；
- 一份 schema evolution 记录；
- 一份 `reports/week04/iceberg_baseline_report.md`。

自检清单

- 我使用的是 `runbooks/week04/README.md` 中的真实命令。
- 我能说明每一步的目标、期望输出、失败排查和验收不变量。
- 我至少让 4 张表进入实验范围。
- 我把实验结果整理成了 report，而不是只截图。
- 我能指出哪个 snapshot 是旧状态，哪个 snapshot 是当前状态。
- 我能说明 add-column 是否破坏旧数据，以及证据在哪里。

课后最小行动

把本实验的观察记录整理到：

`reports/week04/iceberg_baseline_report.md`